COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/7/26	**DATE SUBMITTED:**	9/7/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 7-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

A.1. Bubble Sort Technique

Using the bubble sort technique, sorting is done in passes or iteration. Thus at the end of each iteration, the heaviest element is placed at its proper place in the list. In other words, the largest element in the list bubbles up.

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    //for loop to repeat inner loop
    for(int i = 0; i < arrSize; i++) {
        //inner loop that for k = i
        for(int j = i+1; j < arrSize; j++) {
            if(arr[i] > arr[j]){
                std::swap(arr[j], arr[i]);
            }
        }
    }
}
```

Include a screenshot of your code and output console along with your observations in table 7-2.

A.2. Selection Sort Algorithm

Selection sort is quite a straightforward sorting technique as the technique only involves finding the smallest element in every pass and placing it in the correct position. Selection sort works efficiently when the list to be sorted is of small size but its performance is affected badly as the list to be sorted grows in size. Hence we can say that selection sort is not advisable for larger lists of data.

```
template<typename T>
int routineSmallest(T arr[], int K, const int arrSize) { //array,
current index, arrsize
    int position = K;
    T smallest = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallest){
            smallest = arr[j];
            position = j;
        }
    }
    return position;
}

template<typename T>
void selectionSort(T arr[], size_t arrSize) {
    int pos, pass;
    T temp;
    for(int i = 0; i < arrSize; i++) {
        pos = routineSmallest(arr, i, arrSize);
        temp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = temp;
        pass++;
    }
}
```

Include a screenshot of your code and output console along with your observations in table 7-3.

A.3. Insertion Sort Algorithm

In the insertion sort technique, we start from the second element and compare it with the first element and put it in a proper place. Then we perform this process for the subsequent elements. We compare each element with all its previous elements and put or insert the element in its proper position. Insertion sort technique is more feasible for arrays with a smaller number of elements. It is also useful for sorting linked lists. Linked lists have a pointer to the next element (in case of a singly linked list) and a pointer to the previous element as well (in case of a doubly linked list). Hence it becomes easier to implement insertion sort for a linked list.

```cpp
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    int k = 0, j;
    T temp;

    while(k < arrSize) {
        temp = arr[k];
        j = k-1;

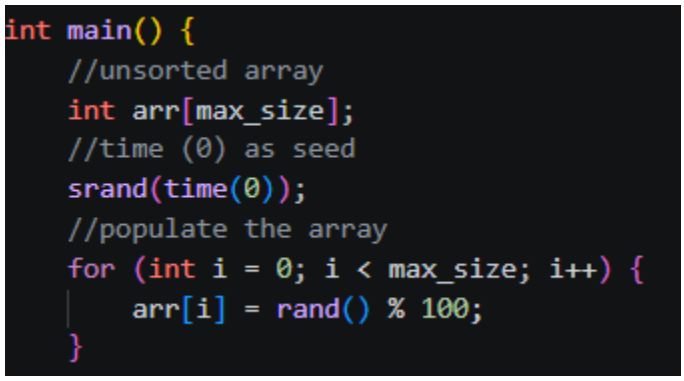
        while(temp < arr[j]){
            arr[j+1] = arr[j];
            j--;
        }

        arr[j+1] = temp;
        k++;
    }

}
```

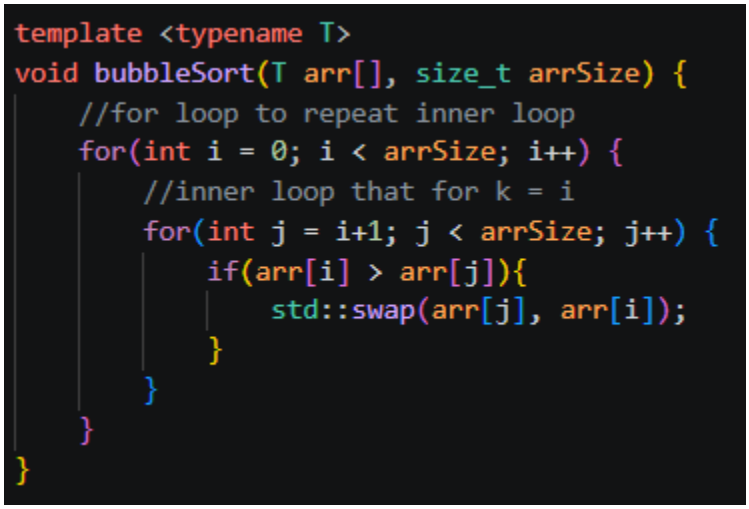
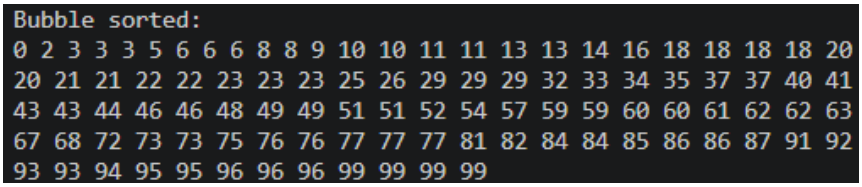
Include a screenshot of your code and output console along with your observations in table 7-4.

Output:

Code + Console Screenshot	 Unsorted: 46 86 91 25 43 29 75 95 29 99 14 10 96 60 73 37 13 6 99 67 3 51 18 87 3 73 13 20 93 93 76 59 92 21 8 6 96 34 8 23 62 68 57 94 35 44 72 11 18 20 81 99 48 9 0 77 49 37 77 76 32 62 18 99 46 23 43 16 63 2 21 96 29 10 52 59 33 41 22 23 51 5 77 60 54 86 6 11 84 3 26 85 95 82 49 22 84 61 18 40
Observations	These lines of code are used to create the data array using srand(time(0)) as a

	seed. It also limits data from range 0 - 100 with the modulo operation. This code has a time complexity of O(n).

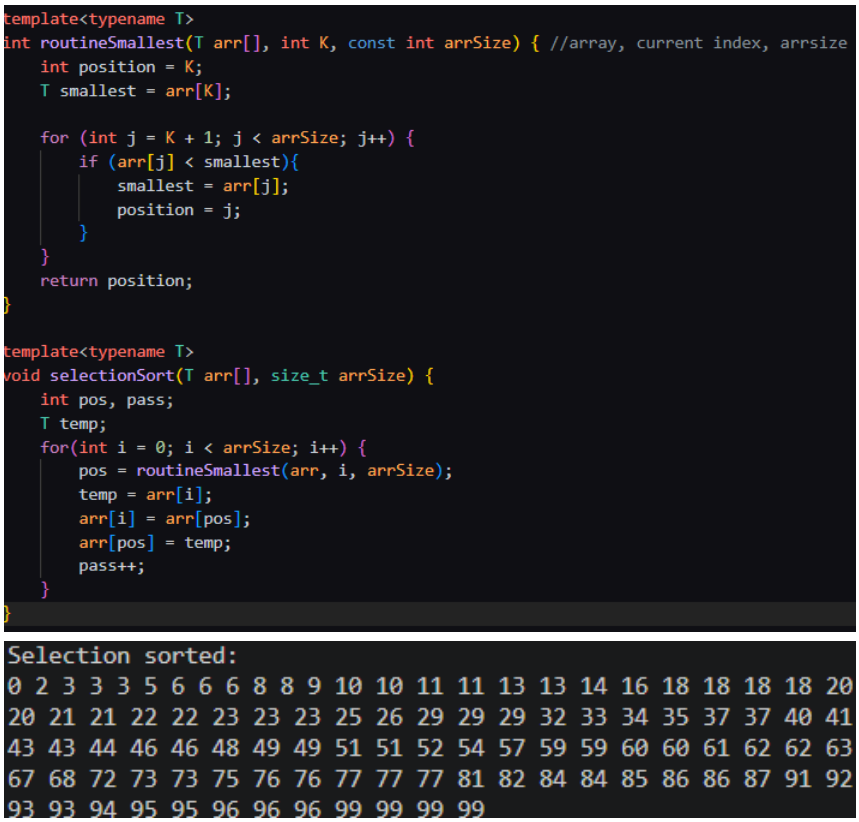
Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	 
Observations	This function runs through each element in the array, comparing it to the one next to it, then swapping if the next value is larger, until the array is sorted. This has a time complexity of $O(n^2)$ since it's a nested for loop. Though if we take the best case scenario—the array is already sorted—it will only have a time complexity of O(n)

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    //for loop to repeat inner loop
    for(int i = 0; i < arrSize; i++) {
        //inner loop that for k = i
        for(int j = i+1; j < arrSize; j++) {
            if(arr[i] > arr[j]){
                std::swap(arr[j], arr[i]);
            }
        }
    }
}
```

```
Bubble sorted:
0 2 3 3 3 5 6 6 6 8 8 9 10 10 11 11 13 13 14 16 18 18 18 18 20
20 21 21 22 22 23 23 23 25 26 29 29 29 32 33 34 35 37 37 40 41
43 43 44 46 46 48 49 49 51 51 52 54 57 59 59 60 60 61 62 62 63
67 68 72 73 73 75 76 76 77 77 77 81 82 84 84 85 86 86 87 91 92
93 93 94 95 95 96 96 96 99 99 99 99
```

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	The selection sort function loops through the whole array index and puts the smallest value on each iteration. It uses a helper function 'routineSmallest" to get the smallest value after the current index and swaps their position. The function has a time complexity of O(n), and the helper function also has a time complexity of O(n). resulting in an $O(n^2)$ time complexity. The best case scenario—array already being sorted—won't change the time complexity since it will still iterate through the whole array and use the whole length of it with the helper function.

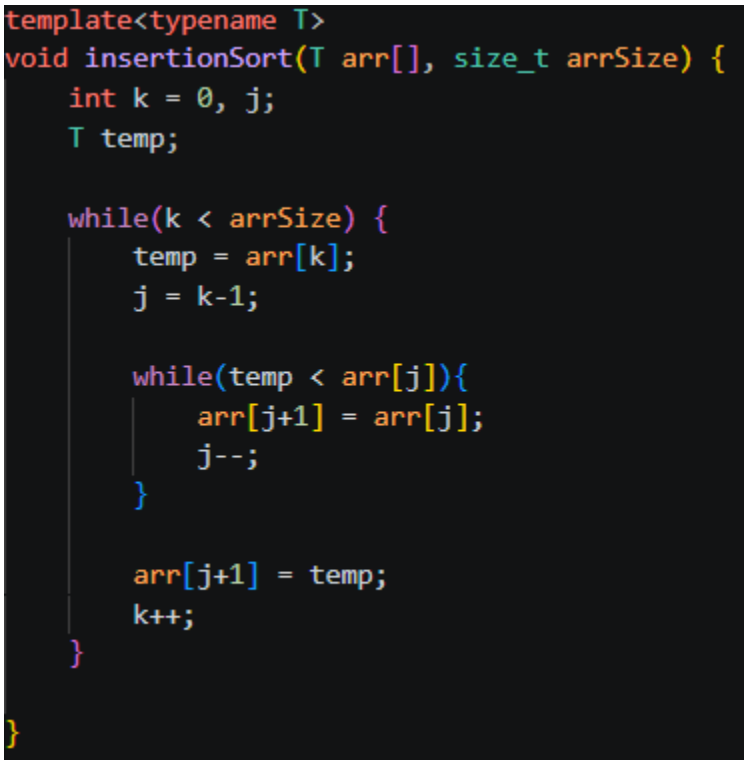
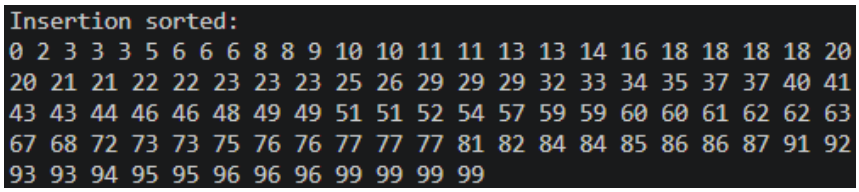
```
template<typename T>
int routineSmallest(T arr[], int K, const int arrSize) { //array, current index, arrsize
    int position = K;
    T smallest = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallest){
            smallest = arr[j];
            position = j;
        }
    }
    return position;
}

template<typename T>
void selectionSort(T arr[], size_t arrSize) {
    int pos, pass;
    T temp;
    for(int i = 0; i < arrSize; i++) {
        pos = routineSmallest(arr, i, arrSize);
        temp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = temp;
        pass++;
    }
}
```

```
Selection sorted:
0 2 3 3 3 5 6 6 6 8 8 9 10 10 11 11 13 13 14 16 18 18 18 18 20
20 21 21 22 22 23 23 23 25 26 29 29 29 32 33 34 35 37 37 40 41
43 43 44 46 46 48 49 49 51 51 52 54 57 59 59 60 60 61 62 62 63
67 68 72 73 73 75 76 76 77 77 77 81 82 84 84 85 86 86 87 91 92
93 93 94 95 95 96 96 96 99 99 99 99
```

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	 
Observations	This function works by getting the first element of an array, then comparing it with the succeeding elements, placing it in its proper position, repeating until all elements are in the correct position. This has a time complexity of O(n). Though if we take the best case scenario—the array is already sorted—it will only have a time complexity of O(n)

```
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    int k = 0, j;
    T temp;

    while(k < arrSize) {
        temp = arr[k];
        j = k-1;

        while(temp < arr[j]){
            arr[j+1] = arr[j];
            j--;
        }

        arr[j+1] = temp;
        k++;
    }

}
```

```
Insertion sorted:
0 2 3 3 3 5 6 6 6 8 8 9 10 10 11 11 13 13 14 16 18 18 18 18 20
20 21 21 22 22 23 23 23 25 26 29 29 29 32 33 34 35 37 37 40 41
43 43 44 46 46 48 49 49 51 51 52 54 57 59 59 60 60 61 62 62 63
67 68 72 73 73 75 76 76 77 77 77 81 82 84 84 85 86 86 87 91 92
93 93 94 95 95 96 96 96 99 99 99 99
```

Table 7-4. Insertion Sort Algorithm

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Pseudocode:

1. **Create array of unsorted integers from 1-5**
2. **Create an array of candidates**
3. **Show unsorted array of votes**
4. **Sort array using insertion sort.**
5. **Count the votes and store in an array**
6. **Compare values in the array, store the highest index**
7. **Check if the winner has a similar no of votes**
8. **Print results**

Output Console Showing Sorted Array	Manual Count Count	Result of Algorithm
Bubble sorted: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	1. 15 2. 19 3. 15 4. 31 5. 20	The winner of the election is Lalla Brielle Yabut, with a total of 31 votes!

Selection sorted: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	1. 20 2. 15 3. 19 4. 21 5. 25	The winner of the election is Franklin Reland Castro, with a total of 25 votes!
Insertion sorted: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	1. 24 2. 18 3. 27 4. 15 5. 16	The winner of the election is Deja Jayla Bañaga, with a total of 27 votes!

- Using all three sorting algorithms resulted in similarly successful results. In my opinion, insertion sort would be the fastest in sorting these kinds of arrays having similar data repeating many times since it already sorts them while it runs, never reaching into the $O(n^2)$ time.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Checking error**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Fixing the logic for checking the winner and if a tie has occurred, I tried to display the tied candidates, but there were too many conditions to consider, so I just resorted to a simple boolean status.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.